AI Lab-2

8-puzzle using DFS

Code:

```
GOAL_STATE = (1, 2, 3, 4, 5, 6, 7, 8, 0)

class PuzzleNode:
    def __init__(self, state, parent=None, action=None, depth=0):
        self.state = state
        self.parent = parent
        self.action = action
        self.depth = depth

    def get_blank_index(self):
        return self.state.index(0)

    def generate_successors(self):
        successors = []
        idx = self.get_blank_index()
        row, col = idx // 3, idx % 3

        moves = [(-1, 0, 'Up'), (1, 0, 'Down'), (0, -1, 'Left'), (0, 1, 'Right')]

        for dr, dc, action in moves:
            new_row, new_col = row + dr, col + dc
            if 0 <= new_row < 3 and 0 <= new_col < 3:
                new_idx = new_row * 3 + new_col
                state_list = list(self.state)
                # Swap elements to create the new board state
                state_list[idx], state_list[new_idx] = state_list[new_idx],
state_list[idx]

                successor_node = PuzzleNode(
                    state=tuple(state_list),
                    parent=self,
                    action=action,
                    depth=self.depth + 1
                )
                successors.append(successor_node)
        return successors

    def get_path(self):
        node, path = self, []
        while node:
            path.append((node.action, node.state))
            node = node.parent
        return path[::-1]

def print_board(state):
    for i in range(0, 9, 3):
        print(f" {state[i]} {state[i+1]} {state[i+2]} ")
```

```
    print("-" * 11)

def depth_first_search(start_state, max_depth_limit=30):
    start_node = PuzzleNode(start_state)
    stack = [start_node]
    visited = set()

    while stack:
        node = stack.pop()

        if node.state == GOAL_STATE:
            return node.get_path()

        if node.depth < max_depth_limit:
            visited.add(node.state)
            # Reverse successors to explore them in standard order
            for successor in reversed(node.generate_successors()):
                if successor.state not in visited:
                    stack.append(successor)
    return None

if __name__ == "__main__":
    # Initial state (3 moves away from goal)
    initial_puzzle = (1, 2, 3, 4, 0, 5, 7, 8, 6)

    print("Initial Board Setup:")
    print_board(initial_puzzle)

    print("\n--- Running Depth First Search (DFS) ---")
    dfs_solution = depth_first_search(initial_puzzle, max_depth_limit=25)

    if dfs_solution:
        print(f"DFS found a solution path in {len(dfs_solution) - 1} moves.")
        for action, state in dfs_solution:
            if action:
                print(f"Action taken: Move blank tile '{action}'")
            print_board(state)
    else:
        print("No solution found within the specified depth limit.")
```

Output:

```
PS C:\Users\Administrator\Desktop\1BM24CS316> & c:\python314\python.exe c:/Users/Administrator/Desktop/1BM24CS316/8puzzdfs.py
Initial Board Setup:
 1 2 3
 4 0 5
 7 8 6
-----------

--- Running Depth First Search (DFS) ---
DFS found a solution path in 24 moves.
 1 2 3
 4 0 5
 7 8 6
-----------
Action taken: Move blank tile 'Up'
 1 0 3
 4 2 5
 7 8 6
-----------
Action taken: Move blank tile 'Right'
 1 3 0
 4 2 5
 7 8 6
-----------
Action taken: Move blank tile 'Down'
 1 3 5
 4 2 0
 7 8 6
-----------
Action taken: Move blank tile 'Down'
 1 3 5
 4 2 6
 7 8 0
-----------
Action taken: Move blank tile 'Left'
 1 3 5
 4 2 6
 7 0 8
-----------
Action taken: Move blank tile 'Up'
 1 3 5
 4 0 6
 7 2 8
-----------
Action taken: Move blank tile 'Up'
 1 0 5
 4 3 6
 7 2 8
-----------
Action taken: Move blank tile 'Left'
 0 1 5
 4 3 6
 7 2 8
```

```
-----------
Action taken: Move blank tile 'Down'
 4 1 5
 0 3 6
 7 2 8
-----------
Action taken: Move blank tile 'Right'
 4 1 5
 3 0 6
 7 2 8
-----------
Action taken: Move blank tile 'Down'
 4 1 5
 3 2 6
 7 0 8
-----------
Action taken: Move blank tile 'Right'
 4 1 5
 3 2 6
 7 8 0
-----------
Action taken: Move blank tile 'Up'
 4 1 5
 3 2 0
 7 8 6
-----------
Action taken: Move blank tile 'Left'
 4 1 5
 3 0 2
 7 8 6
-----------
Action taken: Move blank tile 'Left'
 4 1 5
 0 3 2
 7 8 6
-----------
Action taken: Move blank tile 'Up'
 0 1 5
 4 3 2
 7 8 6
-----------
Action taken: Move blank tile 'Right'
 1 0 5
 4 3 2
 7 8 6
-----------
Action taken: Move blank tile 'Right'
 1 5 0
 4 3 2
 7 8 6
```

```
Action taken: Move blank tile 'Down'
 1 5 2
 4 3 0
 7 8 6
-----------
Action taken: Move blank tile 'Left'
 1 5 2
 4 0 3
 7 8 6
-----------
Action taken: Move blank tile 'Up'
 1 0 2
 4 5 3
 7 8 6
-----------
Action taken: Move blank tile 'Right'
 1 2 0
 4 5 3
 7 8 6
-----------
Action taken: Move blank tile 'Down'
 1 2 3
 4 5 0
 7 8 6
-----------
Action taken: Move blank tile 'Down'
 1 2 3
 4 5 6
 7 8 0
-----------
PS C:\Users\Administrator\Desktop\1BM24CS316>
```

8-puzzle using IDS

Code:

```
GOAL_STATE = (1, 2, 3, 4, 5, 6, 7, 8, 0)

class PuzzleNode:
    def __init__(self, state, parent=None, action=None, depth=0):
        self.state = state
        self.parent = parent
        self.action = action
        self.depth = depth

    def get_blank_index(self):
        return self.state.index(0)

    def generate_successors(self):
        successors = []
        idx = self.get_blank_index()
        row, col = idx // 3, idx % 3

        moves = [(-1, 0, 'Up'), (1, 0, 'Down'), (0, -1, 'Left'), (0, 1, 'Right')]

        for dr, dc, action in moves:
            new_row, new_col = row + dr, col + dc
            if 0 <= new_row < 3 and 0 <= new_col < 3:
                new_idx = new_row * 3 + new_col
                state_list = list(self.state)
                state_list[idx], state_list[new_idx] = state_list[new_idx],
state_list[idx]

                successor_node = PuzzleNode(
                    state=tuple(state_list),
                    parent=self,
                    action=action,
                    depth=self.depth + 1
                )
                successors.append(successor_node)
        return successors

    def get_path(self):
        node, path = self, []
        while node:
            path.append((node.action, node.state))
            node = node.parent
        return path[::-1]

def print_board(state):
    for i in range(0, 9, 3):
        print(f" {state[i]} {state[i+1]} {state[i+2]} ")
    print("-" * 11)

def depth_limited_search(node, goal, limit, visited_path):
```

```
    if node.state == goal:
        return node
    if node.depth >= limit:
        return None

    visited_path.add(node.state)
    for successor in node.generate_successors():
        if successor.state not in visited_path:
            result = depth_limited_search(successor, goal, limit, visited_path)
            if result is not None:
                return result

    visited_path.remove(node.state)
    return None

def iterative_deepening_search(start_state, max_limit=50):

    for depth in range(max_limit):
        start_node = PuzzleNode(start_state)
        visited_path = set()
        result_node = depth_limited_search(start_node, GOAL_STATE, depth, visited_path)
        if result_node:
            return result_node.get_path()
    return None

if __name__ == "__main__":
    initial_puzzle = (1, 2, 3, 4, 0, 5, 7, 8, 6)

    print("Initial Board Setup:")
    print_board(initial_puzzle)

    print("\n--- Running Iterative Deepening Search (IDS) ---")
    ids_solution = iterative_deepening_search(initial_puzzle)

    if ids_solution:
        print(f"IDS found the OPTIMAL solution path in {len(ids_solution) - 1} moves.")
        for action, state in ids_solution:
            if action:
                print(f"Action taken: Move blank tile '{action}'")
            print_board(state)
    else:
        print("No solution found within the specified iteration limits.")
```

Output:

```
PS C:\Users\Administrator\Desktop\1BM24CS316> & c:\python314\python.exe c:/Users/Administrator/Desktop/1BM24CS316/8puzzids.py
Initial Board Setup:
 1 2 3
 4 0 5
 7 8 6
----------

--- Running Iterative Deepening Search (IDS) ---
IDS found the OPTIMAL solution path in 2 moves.
 1 2 3
 4 0 5
 7 8 6
----------
Action taken: Move blank tile 'Right'
 1 2 3
 4 5 0
 7 8 6
----------
Action taken: Move blank tile 'Down'
 1 2 3
 4 5 6
 7 8 0
----------
```